Lekcja 5. Preprocesor i dyrektywy - rozwiązania

A. Odpowiedzi do pytań kontrolnych

1. **1.** Preprocesor to etap przetwarzania kodu źródłowego przed kompilacją. Dołącza pliki, rozwija makra i decyduje o włączeniu lub pominięciu fragmentów kodu.
2. **2.** Dyrektywa zaczyna się od # i jest poleceniem dla preprocesora. Nie jest wykonywaną instrukcją programu.
3. **3.** Nawiasy ostre oznaczają zwykle plik z biblioteki standardowej lub katalogów systemowych. Cudzysłów oznacza najczęściej plik użytkownika szukany w projekcie.
4. **4.** Header guard chroni plik nagłówkowy przed wielokrotnym dołączeniem i błędami powtórzonych deklaracji/definicji.
5. **5.** #define może definiować stałe tekstowe, aliasy, makra funkcyjne i większe fragmenty kodu.
6. **6.** constexpr ma typ danych, jest sprawdzany przez kompilator i lepiej współpracuje z diagnostyką błędów.
7. **7.** #undef usuwa definicję makra.
8. **8.** #ifdef sprawdza, czy makro jest zdefiniowane, a #ifndef - czy nie jest zdefiniowane.
9. **9.** W trybie DEBUG można dołączać dodatkowe komunikaty diagnostyczne, testy pomocnicze albo pomiary.
10. **10.** Makra są podstawieniami tekstowymi, nie kontrolują typów i mogą wielokrotnie używać argumentów.
11. **11.** IDE to zintegrowane środowisko programistyczne.
12. **12.** API to interfejs programistyczny, czyli zestaw reguł i funkcji do korzystania z danego komponentu.

B. Analiza kodu - przykładowe odpowiedzi

B1. Kompilator zobaczy równoważny tekst:

```
double pole = 3.14 * 5 * 5;
```

B2. Po rozwinięciu makra argument a++ pojawia się dwa razy, więc zmienna może zostać zwiększona dwukrotnie. To typowa pułapka makrofunkcji.

```
int wynik = ((a++) * (a++));
```

B3. Do kompilacji trafi komunikat „Tryb diagnostyczny”, ponieważ makro DEBUG zostało wcześniej zdefiniowane.

C. Przykładowe rozwiązania programistyczne

Zadanie 1

```
#include <iostream>
#include <cmath>
#include <string>

int main() {
    std::string imie;
    double liczba;

    std::cout << "Podaj imie: ";
    std::cin >> imie;
    std::cout << "Podaj liczbe nieujemna: ";
    std::cin >> liczba;
```

```
    if (liczba >= 0) {
        std::cout << "Witaj, " << imie << "!" << std::endl;
        std::cout << "Pierwiastek: " << std::sqrt(liczba) << std::endl;
    } else {
        std::cout << "Nie mozna obliczyc pierwiastka z liczby ujemnej." << std::endl;
    }
}
```

Zadanie 2

```
// jednostki.h
#ifndef JEDNOSTKI_H
#define JEDNOSTKI_H

double kmNaMile(double km);
double kgNaFunty(double kg);
double litryNaGalony(double litry);

#endif
// jednostki.cpp
#include "jednostki.h"

double kmNaMile(double km) { return km * 0.621371; }
double kgNaFunty(double kg) { return kg * 2.20462; }
double litryNaGalony(double litry) { return litry * 0.264172; }
// main.cpp
#include <iostream>
#include "jednostki.h"

int main() {
    std::cout << "10 km = " << kmNaMile(10) << " mil" << std::endl;
    std::cout << "5 kg = " << kgNaFunty(5) << " funtow" << std::endl;
    std::cout << "8 l = " << litryNaGalony(8) << " galonow" << std::endl;
}
```

Zadanie 3

```
// Wersja z #define
#include <iostream>
#define PI 3.14159265
int main() {
    double r = 4;
    std::cout << PI * r * r;
}
// Wersja zalecana w C++
#include <iostream>
int main() {
    constexpr double PI = 3.14159265;
    double r = 4;
    std::cout << PI * r * r;
}
```

Wersja z constexpr jest bezpieczniejsza, bo PI ma typ double i jest sprawdzane przez kompilator.

Zadanie 4

```
#include <iostream>
#define DEBUG

int main() {
    int a = 10, b = 5;
```

```
#ifdef DEBUG
    std::cout << "[DEBUG] a=" << a << ", b=" << b << std::endl;
#endif

    int suma = a + b;

#ifdef DEBUG
    std::cout << "[DEBUG] Obliczono sume" << std::endl;
#endif

    std::cout << "Wynik: " << suma << std::endl;
}
```

Zadanie 5

```
// matematyka.h
#ifndef MATEMATYKA_H
#define MATEMATYKA_H

int kwadrat(int x);

#endif
// matematyka.cpp
#include "matematyka.h"
int kwadrat(int x) { return x * x; }
// main.cpp
#include <iostream>
#include "matematyka.h"
#include "matematyka.h"

int main() {
    std::cout << kwadrat(7) << std::endl;
}
```

Zadanie 6

```
#include <iostream>

#define USE_MACROS

#ifdef USE_MACROS
#define POLE(a,b) ((a) * (b))
#define OBWOD(a,b) (2 * ((a) + (b)))
#else
int pole(int a, int b) { return a * b; }
int obwod(int a, int b) { return 2 * (a + b); }
#endif

int main() {
    int a, b;
    std::cout << "Podaj boki prostokata: ";
    std::cin >> a >> b;

#ifdef USE_MACROS
    std::cout << "Pole: " << POLE(a,b) << std::endl;
    std::cout << "Obwod: " << OBWOD(a,b) << std::endl;
#else
    std::cout << "Pole: " << pole(a,b) << std::endl;
    std::cout << "Obwod: " << obwod(a,b) << std::endl;
#endif
}
```

Uwagi metodyczne

- W zadaniach praktycznych warto wymagać od ucznia krótkiego komentarza: dlaczego użył danej dyrektywy.
- Przy ocenianiu należy zwrócić uwagę na podział projektu na pliki, czytelność nazw oraz poprawność header guard.
- Zadanie 6 dobrze nadaje się do omówienia różnicy między kodem aktywnym a kodem pominiętym przez preprocesor.